

Monge 性与在线决策单调性 DP 优化： 从 SMAWK 到 LARSCH 及其在算法竞赛中的应用

杨晨

25210980120

算法课程期末综述论文

日期：2026 年 6 月 20 日

摘 要

在近期 AtCoder、Codeforces 与洛谷的若干题解和模板文章中，一个反复出现的现象是：不少实现范式并非全新算法，而是经典矩阵搜索与动态规划加速理论在竞赛约束下的再组织。Monge 性、totally monotone 矩阵、SMAWK 算法与 LARSCH 算法构成了这样一条技术脉络：它们把形如 $dp_i = \min_{j < i} \{dp_j + w(j, i)\}$ 的二次动态规划降到近线性甚至线性复杂度。内容围绕 Monge 矩阵、全单调矩阵与决策单调性的定义，Knuth-Yao 优化、SMAWK 与在线动态规划的代表性结果，以及一种更容易写进竞赛代码的简化 LARSCH 分治框架展开；简化框架包含明确的算法契约、正确性证明和复杂度分析。应用部分结合 AtCoder、Codeforces 与洛谷近年的题解和教程，讨论 WQS 二分、滑动窗口代价、min-plus/max-plus 卷积与背包优化中的使用方式，并以 ABC355G Baseball 为例展示从题意建模、Monge 性证明到实现选择的完整流程。

关键词：算法竞赛，动态规划优化，Monge 矩阵，SMAWK，LARSCH，决策单调性

1 引言

算法竞赛选手常说的近期技巧大致分为两类。一类是某个题目独有的构造、计数或数据结构技巧；另一类则是经典算法理论在竞赛约束下的再包装。洛谷、Codeforces 与 AtCoder 的若干近期题解中出现的 Monge DP、SMAWK、LARSCH、Aliens trick 与 min-plus convolution，大多属于后一类：理论根基较早，但可写性、适用边界与工程组合方式正在变化。

核心问题是一维在线动态规划

$$dp_i = \min_{0 \leq j < i} \{dp_j + w(j, i)\}, \quad i = 1, 2, \dots, n. \quad (1)$$

朴素计算需要枚举所有二元组 (j, i) ，复杂度为 $O(n^2)$ 。如果代价函数 w 满足 Monge 性，则最优决策点通常具有单调结构，可以使用分治优化、SMAWK 或 LARSCH 等方法降低复杂度。这条技术线索同时具备清楚的数学定义、完整的历史脉络和鲜明的竞赛实践背景。Monge 性可以用一个四点不等式刻画，SMAWK 研究的是全单调矩阵行最值搜索，二者之间存在严格关系；Knuth 对最优二叉搜索树的二次优化、Yao 的四边形不等式框架、Aggarwal 等人的 SMAWK、Larmore 与 Schieber 的在线动态规划，构成了一条自然的算法发展线；AtCoder ABC355 G 的官方题解把分治、SMAWK、LARSCH 与凸包优化并列比较 [3]；Codeforces 上也有 SMAWK 作为分治优化替代方案的教程与讨论 [12]；洛谷 2025 年的算法文章进一步强调简化 LARSCH 在滑动窗口代价函数和莫队代价函数下的实用性 [16]。

结构安排按“定义—历史—算法—应用—现状”展开。第 2 节定义 Monge 性、全单调矩阵与在线 DP。第 3 节回顾历史代表结果。第 4 节给出简化 LARSCH 框架，证明正确性并分析复杂度。第 5 节总结算法竞赛中的应用。第 6 节讨论目前的实践状态、局限和解释框架。

2 问题定义与基本结构

2.1 Monge 矩阵与全单调矩阵

定义 2.1 (Monge 矩阵). 对一个矩阵 A , 如果对任意 $i_1 < i_2$ 与 $j_1 < j_2$ 都有

$$A_{i_1, j_1} + A_{i_2, j_2} \leq A_{i_1, j_2} + A_{i_2, j_1}, \quad (2)$$

则称 A 是用于求最小值的 Monge 矩阵。若不等号方向相反, 则对应求最大值版本。

定义 2.2 (全单调矩阵). 如果矩阵 A 的任意子矩阵中, 逐行最小值所在列的下标都是非降的, 则称 A 是 totally monotone 矩阵。若只要求原矩阵本身的逐行最小值列非降, 则称为单调矩阵。除非特别说明, 行最小值均按统一的最左平局规则选取。

Monge 性比全单调性更容易从具体代价函数中证明。经典结论是: 用于求最小值的 Monge 矩阵一定是 totally monotone。SMAWK 的重要性在于, 它不需要显式存储矩阵; 对 $m \times n$ 的全单调矩阵, 只要可以在需要时计算 $A_{i,j}$, 就能用 $O(m+n)$ 次矩阵查询求出所有行最值 [1]。

SMAWK 可以理解为两个阶段的递归矩阵搜索: 先利用全单调性删去不可能成为行最小值的列, 使列数降到与行数同阶; 再递归求奇数行或偶数行的最小值, 并在相邻已知行之间插值搜索剩余行。这个 reduce/interpolate 思路是后续 Monge DP 优化的离线基准。

定理 2.1 (Monge 蕴含决策单调). 设 A 满足式(2), 并令 p_i 为第 i 行最左的最小值所在列, 则 p_i 关于 i 非降。

证明. 反设存在 $i_1 < i_2$ 但 $p_{i_1} > p_{i_2}$ 。令 $j_1 = p_{i_2}$, $j_2 = p_{i_1}$, 于是 $j_1 < j_2$ 。由 Monge 不等式,

$$A_{i_1, j_1} + A_{i_2, j_2} \leq A_{i_1, j_2} + A_{i_2, j_1}.$$

另一方面, j_2 是第 i_1 行的最左最小值, j_1 是第 i_2 行的最左最小值, 因此

$$A_{i_1, j_2} \leq A_{i_1, j_1}, \quad A_{i_2, j_1} \leq A_{i_2, j_2}.$$

两组不等式相加只能全部取等。若全部相等, 则 j_1 也是第 i_1 行的最小值, 且 $j_1 < j_2 = p_{i_1}$, 这与 p_{i_1} 是最左最小值矛盾。因此 $p_{i_1} \leq p_{i_2}$ 。□

任意子矩阵仍满足同一个 Monge 不等式, 所以同一证明也给出 totally monotone 性; 区别只在于把行、列限制到子矩阵后重新应用最左平局规则。

2.2 从矩阵搜索到动态规划

对式(1), 定义合法单元上的隐式矩阵

$$A_{i,j} = dp_j + w(j, i), \quad 0 \leq j < i \leq n. \quad (3)$$

求 dp_i 就是在第 i 行的合法列 $j < i$ 中找最小值。非法位置 $j \geq i$ 不参与转移; 如果把它们形式上填为 $+\infty$, 得到的是阶梯形矩阵, 而不是普通的满矩形矩阵。后续算法实际需要的是候选前缀上的阶梯全单调性: 固定候选前缀后, 按统一平局规则选出的行最小值列随行号非降。

引理 2.2 (列平移不破坏 Monge 性). 若代价函数满足

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c), \quad a < b < c < d,$$

则矩阵 $A_{i,j} = dp_j + w(j, i)$ 在所有四个单元均合法的二行二列上仍满足 Monge 不等式。

证明. 取 $j_1 < j_2 < i_1 < i_2$ 。由 w 的 Monge 性,

$$w(j_1, i_1) + w(j_2, i_2) \leq w(j_1, i_2) + w(j_2, i_1).$$

两边同时加上 $dp_{j_1} + dp_{j_2}$, 得到

$$A_{i_1, j_1} + A_{i_2, j_2} \leq A_{i_1, j_2} + A_{i_2, j_1}.$$

列项 dp_j 在二行二列比较中完全抵消, 因此不会改变 Monge 性。合法域的阶梯形限制只减少可比较的单元; 实际算法从不查询非法位置。□

引理 2.3 (候选前缀上的阶梯全单调性). 在式(3) 的合法阶梯区域上, 若所有合法二行二列都满足 *Monge* 不等式, 并且每行平局时取最左最小列, 则对任意候选前缀 t ,

$$q_i^{(t)} = \arg \min_{0 \leq j \leq t, j < i} A_{i,j}$$

随行号 i 非降。

证明. 取 $i_1 < i_2$, 并设 $x = q_{i_1}^{(t)}$ 、 $y = q_{i_2}^{(t)}$ 。若 $x > y$, 则 $y < x < i_1 < i_2$, 所以 x, y 对两行都合法。由 *Monge* 不等式,

$$A_{i_1,y} + A_{i_2,x} \leq A_{i_1,x} + A_{i_2,y}.$$

又因为 x 是第 i_1 行在该前缀中的最左最小列, y 是第 i_2 行在该前缀中的最左最小列, 有

$$A_{i_1,x} \leq A_{i_1,y}, \quad A_{i_2,y} \leq A_{i_2,x}.$$

两组不等式只能全部取等。于是 y 也是第 i_1 行的最小列, 并且 $y < x$, 与 x 的最左性矛盾。因此 $x \leq y$ 。若 $y \geq i_1$, 则本来就有 $x < i_1 \leq y$, 也不会出现逆序。结论成立。□

这两个引理解释了为什么代价函数的 *Monge* 性足以传递到动态规划矩阵, 并进一步给出每个候选前缀上的阶梯全单调性。SMAWK 适用于离线矩阵行最值搜索; 但在式(1) 中, $A_{i,j}$ 又包含尚未全部求出的 dp_j , 矩阵元素只能按下标从小到大的顺序逐步变得可用。这就是在线矩阵搜索与普通离线矩阵搜索的区别, 也是 LARSCH 相比直接 SMAWK 更自然的地方 [13, 14]。

2.3 与常见竞赛优化的关系

常见竞赛模板之间的关系可用图 1 概括。分治优化只需要最优决策单调, 因此要求较弱, 但通常是 $O(n \log n)$ 或带额外层数; SMAWK 要求全单调性, 能在线下矩阵中做到线性; LARSCH 面向在线 DP, 理论上也可做到线性。简化 LARSCH 则牺牲一个对数因子, 换取更低实现难度和更好的滑动代价兼容性。

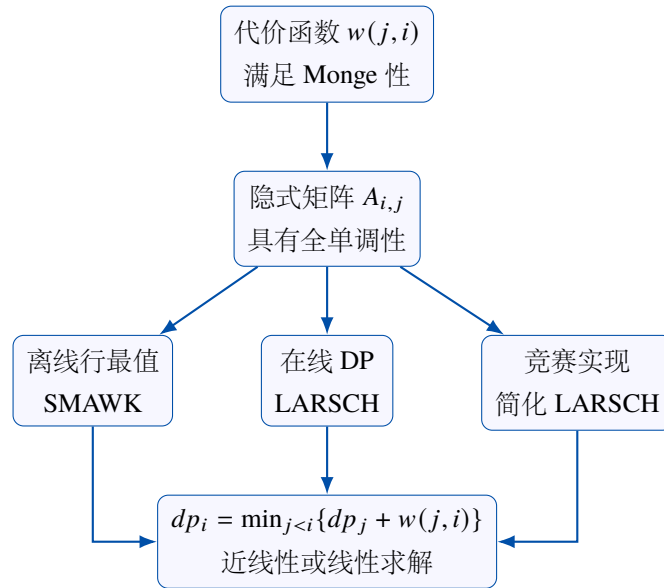


图 1: Monge 性、全单调矩阵搜索与在线 DP 优化的关系。

3 历史代表结果

表 1 按时间列出 *Monge* DP 优化的代表结果。近期竞赛材料中的许多技巧并不是孤立出现的, 而是把 20 世纪 70 年代到 90 年代的动态规划加速理论重新用于现代 OJ 题。

从算法发展脉络看, 历史工作、理论定义、现代竞赛应用之间可以自然衔接。Knuth-Yao 优化解释“为什么决策单调有用”, SMAWK 解释“全单调矩阵可以怎样搜索”, LARSCH 则解释“在线 DP 为什么比离线矩阵搜索难”。除了竞赛中的模板化传播, *Monge* 矩阵和 totally monotone matrix searching 在计算几何、序

表 1: Monge DP 优化相关代表结果。

时间	代表工作	作用
1971	Knuth 最优二叉搜索树 [11]	利用最优根的单调性，将一类区间 DP 从三次降到二次。
1980	Yao 四边形不等式 [17]	给出更一般的 <i>quadrangle inequality</i> 条件，解释何时可做 Knuth-Yao 优化。
1987	SMAWK[1]	对全单调矩阵做隐式行最值搜索，是 Monge DP 离线优化的核心工具。
1991	Larmore-Schieber 在线 DP[13]	将全单调矩阵搜索扩展到在线动态规划，并应用于 RNA 二级结构预测。
1992	稀疏与凸性 DP[7, 8, 10]	系统研究稀疏、凸性和凹性结构下的动态规划加速，展示 Monge 型结构与序列优化之间的联系。
1996	Monge 性综述 [5]	从优化角度综述 Monge 性在排序、匹配、运输和矩阵搜索等问题中的作用。
2009	Knuth-Yao 与全单调性的统一 [4]	证明 Knuth-Yao 加速可视为全单调性的结果，并给出在线版本解释。
2021-2025	竞赛模板化与再传播 [14-16]	将 LARSCH 和简化 LARSCH 写成可复用 C++ 模板，用于 WQS 二分、滑动窗口代价与莫队代价。

列优化、最短路、RNA 二级结构预测等问题中也长期作为基础工具出现 [5, 7, 8, 10, 13]。这里重点放在在线 DP 与竞赛实现，因此不展开所有应用；这些正式文献说明，这条技术线并非只服务于 OJ 题，而是有更广的算法背景。

4 一个核心算法：简化 LARSCH 分治框架

4.1 算法目标与假设

完整 LARSCH 可以在线性时间求解在线 Monge DP，但实现较复杂。竞赛中更常见的是简化 LARSCH：多一个对数因子，代码明显更短，也更容易和滑动窗口或莫队维护的代价函数结合 [15, 16]。

继续考虑式(1)。为了让证明条件精确，记

$$q_i^{(t)} = \arg \min_{0 \leq j \leq t, j < i} \{dp_j + w(j, i)\}$$

为第 i 行在候选前缀 $0, 1, \dots, t$ 中的最左最优列。算法需要的不是单纯的全局决策单调，而是每个候选前缀上的阶梯全单调性：对任意固定 t ，只要 $i_1 < i_2$ 且两行都存在合法候选，就有 $q_{i_1}^{(t)} \leq q_{i_2}^{(t)}$ 。在常见 Monge 代价下，列平移引理和引理 2.3 给出了这个条件。

4.2 伪代码

以下骨架可直接翻译成 C++ 模板。状态为 $0, 1, \dots, n$ ，另设终点哨兵 $N = n + 1$ ，因此数组长度必须为 $N + 1 = n + 2$ 。函数 `relax` 只在严格变优时更新，平局保留更早写入的决策，从而固定为“取最左最优”；实际实现还应按题目数据范围设置足够大的 `INF`，并避免加法溢出。

```
const long long INF = (1LL << 62);
int N = n + 1; // sentinel target
vector<long long> dp(N + 1, INF);
vector<int> pre(N + 1, 0);

void relax(int i, int j) {
```

```

    if (j >= i || dp[j] == INF) return;
    long long cand = dp[j] + cost(j, i);
    if (cand < dp[i]) {           // keep the leftmost optimum on ties
        dp[i] = cand;
        pre[i] = j;
    }
}

void solve(int l, int r) {
    if (r - l == 1) return;
    int mid = (l + r) >> 1;

    for (int j = pre[l]; j <= pre[r]; ++j) {
        relax(mid, j);
    }
    solve(l, mid);

    for (int j = l + 1; j <= mid; ++j) {
        relax(r, j);
    }
    solve(mid, r);
}

dp[0] = 0;
pre[0] = 0;
relax(N, 0);           // only if cost(0,N) is defined
solve(0, N);

```

哨兵 N 的含义由具体问题决定：在最短路径 DP 中，它通常表示终点。若原模型不存在直接边 $(0, N)$ ，证明中可把它建模为代价 $+\infty$ 的形式候选，并令 $pre[N] = 0$ 表示最左的无穷候选；实际代码应使用单独初始化，或在 `relax` 中跳过这条边，而不是调用未定义的 `cost(0, N)`。关键是保持同样的不变量语义，防止无穷值参与加法溢出，并保证所有有效更新都对对应题目中的合法代价。

4.3 正确性证明

引理 4.1 (递归不变量). 调用 $solve(l, r)$ 时满足三条性质：

- (i) $dp_0, dp_1, \dots, dp_l$ 都已经是最终值；
- (ii) dp_r 已经等价地取得候选前缀 $0, 1, \dots, l$ 下的最小值，且 $pre[r] = q_r^{(l)}$ ；
- (iii) $pre[l] = q_l^{(l)}$ 。当 $l = 0$ 时没有合法前驱，约定 $pre[0] = 0$ 。

证明. 初始调用为 $solve(0, N)$ 。此时只有 $dp_0 = 0$ 需要被视为最终值；若直接哨兵边不存在，则把候选 $0 \rightarrow N$ 的代价视为 $+\infty$ 并保留 $pre[N] = 0$ ；否则先用候选 0 更新 N 。两种情况下 N 都已经取得候选前缀 0 下的最小值，并有 $pre[N] = q_N^{(0)}$ ，不变量成立。

设某次调用 $solve(l, r)$ 满足不变量，令 $mid = (l + r)/2$ 。由于 $dp_0, \dots, dp_l$ 都已最终，可以安全计算这些候选对 mid 的贡献；又由 $pre[r] = q_r^{(l)} \leq l$ 可知第一段循环不会访问尚未最终确定的 dp_j 。在候选前缀 $0, 1, \dots, l$ 下，阶梯形全单调性给出

$$q_l^{(l)} \leq q_{mid}^{(l)} \leq q_r^{(l)}.$$

其中 $q_l^{(l)} = pre[l]$, $q_r^{(l)} = pre[r]$ 。第一段循环按递增顺序枚举 $pre[l], pre[l] + 1, \dots, pre[r]$, 因此一定会枚举到 $q_{mid}^{(l)}$; 严格变优才更新的平局规则保留该范围内最左最优列, 从而得到 $pre[mid] = q_{mid}^{(l)}$ 。于是左递归 $solve(1, mid)$ 的不变量成立。

左递归结束后, $dp_0, dp_1, \dots, dp_{mid}$ 都已经成为最终值。第二段循环按递增顺序把新候选 $l+1, l+2, \dots, mid$ 用于更新 r ; 旧候选前缀 $0 \dots l$ 已在调用开始前处理过, 新候选下标都更大, 所以相同代价下保留旧的更左决策仍符合最左规则。循环结束后, r 正好取得候选前缀 $0 \dots mid$ 下的最小值, 并且 $pre[r] = q_r^{(mid)}$ 。左递归结束也使 mid 成为最终状态; 由于 $j = mid$ 对第 mid 行非法, $q_{mid}^{(mid)} = q_{mid}^{(mid-1)} = pre[mid]$ 。于是右递归 $solve(mid, r)$ 的不变量也成立。 $\square$

定理 4.2 (简化 LARSCH 的正确性). 若隐式矩阵在合法阶梯区域上满足上述全单调条件, 且 $relax(i, j)$ 正确计算候选 $dp_j + w(j, i)$, 则算法结束后每个被求解状态的 dp 值均为式(1)的最优值。

证明. 考虑任意叶子调用 $solve(k-1, k)$ 。由递归不变量, 调用该叶子时 dp_k 已经被所有 $0 \leq j \leq k-1$ 的候选更新过, 而这些正是式(1)的全部合法候选。因此 dp_k 已经达到最优。所有状态都会作为某个叶子的右端点出现, 结论成立。 $\square$

4.4 复杂度分析

递归树每一层中, 第二段循环的区间 $[l+1, mid]$ 对不同节点互不相交, 总长度为 $O(n)$ 。第一段循环的区间也可以线性求和: 同一深度的递归节点按端点排成 $x_0 < x_1 < \dots < x_s$, 各节点枚举的列区间为 $[pre[x_a], pre[x_{a+1}]]$ 。这里的 $pre[x_a]$ 是递归不变量下已经得到的候选前缀最左最优列, 不必等同于最终全局最优列。由阶梯全单调性, $pre[x_0], pre[x_1], \dots, pre[x_s]$ 非降; 并且这里的集合扩张始终是候选前缀扩张, 新增候选列下标更大, 平局又保留最左候选, 所以最左最优列只会保持不变或右移。因此这些长度相加至多为 $pre[x_s] - pre[x_0] + s = O(n)$ 。递归深度为 $O(\log n)$, 若 $relax$ 为 $O(1)$, 总复杂度为

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

若代价 $w(j, i)$ 只能用滑动窗口维护, 还需要把每层查询组织成两组单调移动的序列: 第一段循环服务于中点查询, 第二段循环服务于右端点查询。两组状态分开维护时, 每层指针移动仍可控制在 $O(n)$; 若把两种方向混在一个窗口里, 复杂度分析通常不再成立。这也是洛谷文章强调 CF868F 一类问题要拆分查询状态的原因 [16]。

5 算法竞赛中的应用

5.1 WQS 二分与 Aliens trick

很多分段 DP 写作

$$dp_{i,k} = \min_{j < i} \{dp_{j,k-1} + w(j, i)\},$$

其中 k 是段数。WQS 二分, 也常称 Aliens trick, 会把段数限制用惩罚参数 λ 吸收到目标函数中, 变成

$$f_i(\lambda) = \min_{j < i} \{f_j(\lambda) + w(j, i) + \lambda\}.$$

这样每次固定 λ 都只需求解一维在线 Monge DP。实现中通常同时维护二元组“代价、段数”; 一种常用约定是在惩罚后总代价相同的时候取更多段; 随着 λ 增大, 记录的段数不减。二分选择让最优段数跨过目标值的边界参数, 再用惩罚后代价减去 λ 乘目标段数恢复原目标; 若边界处记录的段数并不恰好等于目标段数, 恢复值可理解为离散凸包上该斜率支持线在目标段数处的截距。AtCoder ABC355 G 官方题解明确列出: 分治 + monotone minima 为 $O(N(\log N)^2)$, 分治 + SMAWK 为 $O(N \log N)$, LARSCH 为 $O(N)$, 同时也给出简化 LARSCH 在实现常数上的优势 [3]。

5.2 完整案例：ABC355G Baseball

同样的判断流程可以在 AtCoder ABC355 G Baseball 中完整落地。题目给定权重序列 $P_1, \dots, P_N$ ，先选择 K 个不同位置，再按 P_y 成比例随机产生位置 y ，目标是最小化 y 到最近被选位置的期望距离乘以 $\sum_y P_y$ ；约束为 $N \leq 5 \times 10^4$ ，朴素二次或多维 DP 都不可接受 [2]。

官方题解先把问题改写成有向无环图最短路 [3]。图的顶点为 $0, 1, \dots, N, N+1$ ，其中 0 和 $N+1$ 是左右边界哨兵。边权定义为

$$c(0, j) = \sum_{y=1}^{j-1} P_y(j-y), \quad 1 \leq j \leq N, \quad (4)$$

$$c(i, j) = \sum_{y=i}^{j-1} P_y \min(y-i, j-y), \quad 1 \leq i < j \leq N, \quad (5)$$

$$c(i, N+1) = \sum_{y=i}^N P_y(y-i), \quad 1 \leq i \leq N. \quad (6)$$

原图没有直接边 $(0, N+1)$ ；若为了统一模板形式引入这个位置，也应令 $c(0, N+1) = +\infty$ ，保证它不会参与最优路径。若选择位置为 $x_1 < \dots < x_K$ ，则路径 $0 \rightarrow x_1 \rightarrow \dots \rightarrow x_K \rightarrow N+1$ 的权重正好等于目标值。因此问题等价于求从 0 到 $N+1$ 、恰好使用 $K+1$ 条边的最短路。令 $D_{t,j}$ 表示使用 t 条边到达 j 的最小代价，就得到

$$D_{t,j} = \min_{i < j, (i,j) \in E} \{D_{t-1,i} + c(i,j)\},$$

其中 E 是上面三类边的集合。朴素计算为 $O(N^2K)$ 。

这些边权可以用 P_y 与 $P_y y$ 的前缀和在 $O(1)$ 时间求出。例如令 $S(l, r) = \sum_{y=l}^r P_y$ 、 $T(l, r) = \sum_{y=l}^r P_y y$ ，则

$$c(0, j) = jS(1, j-1) - T(1, j-1), \quad c(i, N+1) = T(i, N) - iS(i, N),$$

而对 $1 \leq i < j \leq N$ ，取 $m = \lfloor (i+j)/2 \rfloor$ ，有

$$c(i, j) = T(i, m) - iS(i, m) + jS(m+1, j-1) - T(m+1, j-1),$$

空区间贡献按 0 处理。

WQS 二分把边数限制吸收到惩罚参数 λ 中：每条边额外加上 λ ，反复求解

$$F_j(\lambda) = \min_{i < j, (i,j) \in E} \{F_i(\lambda) + c(i,j) + \lambda\}, \quad (7)$$

并同时记录最短路使用的边数。实现上同样维护“代价、边数”二元组；可约定惩罚后总代价相同则取更多边，使记录的边数随 λ 增大而不增。题目数据为整数时，可以在整数 λ 上二分搜索使最优边数仍不少于 $K+1$ 的边界参数，或等价地搜索从“不少于”变为“少于”的临界点；在该边界处用 $F_{N+1}(\lambda) - \lambda(K+1)$ 恢复恰好 $K+1$ 条边的原目标值。即使边界最短路本身的记录边数不是 $K+1$ ，离散凸性也保证这个支持线截距给出目标边数处的最优值。官方题解给出的搜索范围为 $O(NS)$ 量级，其中 $S = \sum_i P_i$ [3]。

剩下的关键是证明边权满足 Monge 性。由于 P_y 是非负权重，可以先对每个位置 y 逐点比较贡献，再对 y 求和。对四条相关边均存在的端点 $0 \leq a < b < c < d \leq N+1$ ，需要

$$c(a, c) + c(b, d) \leq c(a, d) + c(b, c). \quad (8)$$

若形式上出现 $(0, N+1)$ ，则 $c(0, N+1) = +\infty$ 位于右侧，式(8)平凡成立；下面只需检查四条边均有限的情况。

先考虑四个端点均在 $1, \dots, N$ 内的情形。固定一个位置 y ，只比较它对式(8)两侧的贡献。若 $a \leq y < b$ ，右侧比左侧多

$$\min(y-a, d-y) - \min(y-a, c-y) \geq 0.$$

若 $b \leq y < c$ ，令 $A = y-a, B = y-b, C = c-y, D = d-y$ 。有 $A \geq B$ 且 $D \geq C$ 。函数 $h(t) = \min(t, D) - \min(t, C)$ 关于 t 单调非降，因此

$$\min(A, D) + \min(B, C) \geq \min(A, C) + \min(B, D).$$

若 $c \leq y < d$ ，右侧比左侧多

$$\min(y - a, d - y) - \min(y - b, d - y) \geq 0.$$

其他位置贡献为零。

边界 $a = 0$ 、 $d \leq N$ 时也可逐点比较右侧减左侧。若 $1 \leq y < b$ ，差为 $d - c \geq 0$ ；若 $b \leq y < c$ ，差为

$$d - c + \min(y - b, c - y) - \min(y - b, d - y) \geq 0,$$

因为把第二个参数从 $c - y$ 增加到 $d - y$ 时， $\min(y - b, \cdot)$ 的增量不超过 $d - c$ ；若 $c \leq y < d$ ，差为

$$d - y - \min(y - b, d - y) \geq 0.$$

边界 $a \geq 1$ 、 $d = N + 1$ 时同理。若 $a \leq y < b$ ，右侧减左侧为

$$y - a - \min(y - a, c - y) \geq 0;$$

若 $b \leq y < c$ ，差为

$$b - a + \min(y - b, c - y) - \min(y - a, c - y) \geq 0,$$

因为 $y - a = (y - b) + (b - a)$ ，而 $\min(\cdot, c - y)$ 的增量不超过 $b - a$ ；若 $c \leq y \leq N$ ，差为 $b - a \geq 0$ 。乘以非负权重 P_y 并对所有 y 求和，即得式(8)。由此，固定 λ 后的式(7)正好落入在线 Monge DP 框架。

表 2 总结了 this 案例的解题链条。关键步骤不是机械套用 LARSCH 模板，而是先将题意转成图模型，再证明边权结构，最后根据在线依赖选择合适的矩阵搜索算法。

表 2: ABC355G Baseball 的 Monge/LARSCH 建模链条。

步骤	内容
题意压缩	选择 K 个位置，最小化带权距离到最近选择点的总和。
图模型	相邻选择点 (i, j) 形成一条边，边权为区间内到较近端点的加权距离；左右哨兵使用单侧距离，且没有有效边 $(0, N + 1)$ 。
段数约束	恰好 $K + 1$ 条边的最短路，用 WQS 二分转为每条边加惩罚 λ 。
Monge 证明	四个端点 $a < b < c < d$ 下逐点比较贡献，得到 $c(a, c) + c(b, d) \leq c(a, d) + c(b, c)$ 。
在线依赖	F_j 依赖更小下标的 F_i ，不能把整张隐式矩阵直接交给离线 SMAWK。
实现选择	前缀和使边权 $O(1)$ 查询；简化 LARSCH 每次评估 λ 为 $O(N \log N)$ ，总复杂度为 $O(N \log N \log(NS))$ 。

5.3 滑动窗口代价与莫队代价

有些转移代价 $w(l, r)$ 不容易用闭式公式 $O(1)$ 计算，但可以在移动左右端点时维护。例如区间内相等数对数量、合法括号子串数量等。这时，普通分治优化虽然知道候选区间，却不一定能安排代价查询顺序；SMAWK 要求随机访问隐式矩阵元素，也会破坏滑动窗口维护。简化 LARSCH 的优势在于递归每层可以把查询组织成两套单调移动的窗口。洛谷的 2025 年文章特别强调，若使用莫队维护代价，第一步和第三步应分开维护两个窗口，否则指针会来回跳动导致复杂度退化 [16]。

5.4 min-plus/max-plus 卷积与背包优化

另一个常见入口是卷积。给定两个数组 A, B ，max-plus 卷积定义为

$$C_i = \max_{j+k=i} (A_j + B_k).$$

一般情形很难突破二次复杂度，但如果至少一边满足与 min/max 方向相容的凸性或凹性，问题可以转化为 Monge 矩阵行最值搜索，使用 SMAWK 或斜率合并得到线性或近线性算法。Codeforces 上关于背包、子集和与 max-plus convolution 的教程明确将“凸数组上的 max-plus convolution 可由 SMAWK 线性求解”列为核心工具 [9]。2026 年 Codeforces 上基于 THUPC 的题解也展示了把 DP 更新理解为 min-plus convolution，并利用凸函数斜率集合压缩状态的思路 [6]。

5.5 与 C++ 模板化的关系

从 C++ 竞赛实践看，这类算法的真正门槛往往不是定理本身，而是三个工程问题：

- (1) 如何证明题目中的 w 确实满足 Monge 性；
- (2) 如何在不破坏复杂度的顺序下查询 w ；
- (3) 如何在平局时稳定选择最左或最右决策，保证单调性。

因此，竞赛模板不应只保存代码，还应保存“证明检查表”。对应检查表见表 3。

表 3: 在竞赛中判断是否可用 Monge/LARSCH 的检查表。

检查项	需要确认的内容
转移形式	是否能写成 $dp_i = \min_{j < i} \{dp_j + w(j, i)\}$ ，或经参数化后化为同形。
Monge 证明	是否能证明四点不等式，且方向、下标顺序与题目代价一致。
在线依赖	若 $A_{i,j}$ 含尚未计算的 dp ，就不能直接离线 SMAWK。
代价访问	w 是 $O(1)$ 、前缀和、滑动窗口还是莫队；查询顺序是否可维护。
平局规则	最优决策固定取最左或最右，并与单调性证明一致。
实现选择	离线矩阵用 SMAWK；在线 DP 用简化 LARSCH；特化函数族可用凸包或 Li Chao tree。

6 State-of-the-art、局限与实践判断

6.1 理论 SOTA 与实践 SOTA

这里的 state-of-the-art 需要分成两个语境。理论 SOTA 关注渐进复杂度和矩阵查询次数：对离线 totally monotone 矩阵，SMAWK 已达到线性数量级查询 [1]；对在线 Monge DP，完整 LARSCH 也达到 $O(n)$ [13, 14]。从这个角度看，线性查询或线性时间是最清楚的理论基准。

实践 SOTA 则不同。算法竞赛中的“最优”还要考虑代码长度、常数、调试难度、平局规则以及代价函数能否按指定顺序访问。完整 LARSCH 虽然渐进最优，但手写难度高；简化 LARSCH 牺牲一个对数因子，得到更短的代码和更稳定的查询顺序，尤其适合滑动窗口或莫队代价。AtCoder 官方题解也提到，虽然 LARSCH 有 $O(N)$ 解法，但简化版 $O(N \log N)$ 在作者实现中常数更低，实际运行时间可能更好 [3]。

表 4 把几种常见方案放在同一张表中。理论最优关注矩阵查询次数或单次在线 DP 的渐进复杂度；竞赛实现则按代价访问方式在简化 LARSCH、SMAWK、monotone minima 与特化凸包之间取舍。

表 4: Monge DP 相关方法的理论与实践对照。

方法	适用条件	复杂度	实现风险
分治优化	全局决策单调，按层离线求值	约 $O(n \log n)$	条件较弱，但无法处理强在线依赖。
Monotone minima	行最优列单调，可随机访问元素	约 $O(n \log n)$	代码简单，常数小；随机访问可能破坏滑动代价。
SMAWK	全单调矩阵，可随机访问元素	$O(n)$ 查询	渐进最优，代码和调试成本较高。
完整 LARSCH	在线 DP 的阶梯全单调矩阵	$O(n)$	理论最优，实现复杂，竞赛中较少手写。
简化 LARSCH	在线 DP，允许一个对数因子	$O(n \log n)$	代码较短，适合与滑动窗口或莫队代价结合。
凸包/Li Chao	函数交点单调或线性函数族	通常 $O(n \log n)$	依赖题目特化结构，不是通用 Monge 模板。

6.2 主要局限

这一技术线也有明显局限。

第一，**Monge 性证明高度依赖题目结构**。很多题并不会直接告诉选手 w 满足四边形不等式，需要通过几何、排序、交换论证或凸性推导得到。一旦证明方向错误，仅掌握模板也无法保证正确性。

第二，一般 **min-plus convolution** 不在这条模板线内。**Monge/SMAWK** 的加速依赖特殊结构。若两个数组都没有凸性、凹性或类似单调结构，就不能直接套用本综述讨论的线性或近线性矩阵搜索技巧。

第三，**工程实现容易被访问顺序破坏**。如果 w 只能滑动维护，随机访问会让本来正确的渐进分析失效。洛谷文章中对 CF868F 的提醒就是典型例子：两个方向的查询必须分开维护，不能共用同一个莫队状态 [16]。

6.3 本文的综合诠释：在线 Monge DP 的四层判定框架

本文的一个综合性观点是：**Monge DP 优化不应被理解为若干孤立模板，而应被理解为“结构证明—矩阵搜索—在线依赖—代价访问”的四层判定流程**。这类方法的价值不只在于某个模板本身，而在于提供了一套可复用的判断流程：

- (1) 先把“决策单调性 DP 优化”改写成“隐式矩阵行最值搜索”；
- (2) 再区分 **SMAWK** 与 **LARSCH** 的核心差异：前者要求离线随机访问，后者处理在线依赖；
- (3) 然后根据代价访问方式选择完整 **LARSCH**、简化 **LARSCH**、**monotone minima** 或特化凸包；
- (4) 最后用表 3 与表 2 这样的检查表，把证明条件和实现风险逐项核对。

在 ACM/OJ 的语境下，这种解释比直接背诵 **SMAWK** 的 **reduce** 步骤更贴近解题过程。熟悉的 $dp_i = \min_j$ 转移只是入口；更关键的是候选区间为什么可以缩小、在线依赖为什么阻止直接离线搜索、以及代价函数能否按所需顺序被高效访问。

7 结论

Monge 性、**SMAWK** 与 **LARSCH** 并不是 2025-2026 年才出现的新算法；近期竞赛题解和模板文章中的新价值，主要体现在这些经典工具被重新组合到在线 DP、WQS 二分和滑动代价维护之中。单纯掌握“分治优化 DP”不足以覆盖这类题目；更关键的是 **Monge 不等式** 能否成立、在线依赖如何处理、代价函数能否按所需顺序访问。

ABC355G 案例说明，**Monge 优化** 不是孤立模板，而是从题意建模、结构证明到实现取舍的一套分析流程。竞赛中看似经验化的优化，往往可以回到 **Monge 不等式** 和隐式矩阵搜索来解释；把在线依赖、代价访问顺序和平局规则合在同一张检查表中，也使 **SMAWK**、**LARSCH** 与简化实现之间的取舍具有清晰的数学依据。

参考文献

- [1] Alok Aggarwal et al. “Geometric applications of a matrix-searching algorithm”. In: *Algorithmica* 2.1–4 (1987), pp. 195–208. doi: [10.1007/BF01840359](https://doi.org/10.1007/BF01840359).
- [2] AtCoder. *G - Baseball: Tokio Marine & Nichido Fire Insurance Programming Contest 2024*. AtCoder. 2024. URL: https://atcoder.jp/contests/abc355/tasks/abc355_g.
- [3] AtCoder. *Official editorial for Problem G: Baseball, Tokio Marine & Nichido Fire Insurance Programming Contest 2024*. AtCoder. 2024. URL: <https://atcoder.jp/contests/abc355/editorial/10209>.
- [4] Wolfgang W. Bein et al. “The Knuth-Yao quadrangle-inequality speedup is a consequence of total monotonicity”. In: *ACM Transactions on Algorithms* 6.1 (2009), 17:1–17:22. doi: [10.1145/1644015.1644032](https://doi.org/10.1145/1644015.1644032).
- [5] Rainer E. Burkard, Bettina Klinz, and Rüdiger Rudolf. “Perspectives of Monge properties in optimization”. In: *Discrete Applied Mathematics* 70.2 (1996), pp. 95–161. doi: [10.1016/0166-218X\(95\)00103-X](https://doi.org/10.1016/0166-218X(95)00103-X).
- [6] dapingguo8. *Codeforces Round 1092 (Div. 1, Div. 2, Based on THUPC 2026 Finals) Editorial*. Codeforces. 2026. URL: <https://codeforces.com/blog/entry/152930>.
- [7] David Eppstein et al. “Sparse dynamic programming I: Linear cost functions”. In: *Journal of the ACM* 39.3 (1992), pp. 519–545. doi: [10.1145/146637.146650](https://doi.org/10.1145/146637.146650).
- [8] David Eppstein et al. “Sparse dynamic programming II: Convex and concave cost functions”. In: *Journal of the ACM* 39.3 (1992), pp. 546–567. doi: [10.1145/146637.146656](https://doi.org/10.1145/146637.146656).
- [9] errorgorn. *[Tutorial] Knapsack, Subset Sum and the (max,+) Convolution*. Codeforces. Jan. 3, 2022. URL: <https://codeforces.com/blog/entry/98663>.
- [10] Zvi Galil and Kunsoo Park. “Dynamic programming with convexity, concavity and sparsity”. In: *Theoretical Computer Science* 92.1 (1992), pp. 49–76. doi: [10.1016/0304-3975\(92\)90135-3](https://doi.org/10.1016/0304-3975(92)90135-3).
- [11] Donald E. Knuth. “Optimum binary search trees”. In: *Acta Informatica* 1.1 (1971), pp. 14–25. doi: [10.1007/BF00264289](https://doi.org/10.1007/BF00264289).
- [12] ko_osaga. *SMAWK algorithm as an alternative for D&C optimization*. Codeforces. Jan. 1, 2023. URL: <https://codeforces.com/blog/entry/110844>.
- [13] Lawrence L. Larmore and Baruch Schieber. “On-line dynamic programming with applications to the prediction of RNA secondary structure”. In: *Journal of Algorithms* 12.3 (1991), pp. 490–515. doi: [10.1016/0196-6774\(91\)90016-R](https://doi.org/10.1016/0196-6774(91)90016-R).
- [14] noshi91. *LARSCH Algorithm*. noshi91/Library. 2021. URL: <https://noshi91.github.io/Library/algorithm/larsch.cpp.html>.
- [15] noshi91. 簡易版 *LARSCH Algorithm*. Hatena Blog. Feb. 18, 2023. URL: <https://noshi91.hatenablog.com/entry/2023/02/18/005856>.
- [16] Register_int. 在线决策单调性地皮还能单老哥分治做? Luogu. Aug. 3, 2025. URL: <https://www.luogu.com.cn/article/vqf42hah>.
- [17] F. Frances Yao. “Efficient dynamic programming using quadrangle inequalities”. In: *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*. STOC ’80. ACM Press, 1980, pp. 429–435. doi: [10.1145/800141.804691](https://doi.org/10.1145/800141.804691).